Building Segmentation in Satellite Images

Second Approach Report

Project ID: 22

Renata Salikhzyanova, Kristina Soloveva,
Elena Tuturina, Irina Bychkova

October 19, 2026

1. Project Overview

Our project focuses on binary building segmentation in aerial images. The baseline is a U-Net with an ImageNet-pretrained ResNet18 encoder and `BCEWithLogitsLoss`. For the second milestone, we investigate whether adding soft Dice improves building overlap with the same data, architecture and training settings.

2. New Approach and Motivation

The second condition replaces the baseline objective with **BCE + Dice loss**. BCE optimizes pixel classification, while Dice rewards overlap between predictions and building masks. The expected benefit is better foreground segmentation despite building/background imbalance. This improvement is a hypothesis to be tested.

For a batch of N patches, the implemented objective is

$$\mathcal{L} = \mathcal{L}_{\text{BCEWithLogits}} + \frac{1}{N}\sum_{n=1}^{N}\left(1 - \frac{2\sum_i p_{ni}y_{ni} + 1}{\sum_i p_{ni} + \sum_i y_{ni} + 1}\right), \qquad p_{ni} = \text{sigmoid}(z_{ni}).$$

BCE is averaged over pixels. Soft Dice is computed per patch, including empty masks, then averaged across the batch. Both coefficients are 1; Dice smoothing is 1 in numerator and denominator. No threshold is applied in the loss.

3. Implementation and Training

We reused the Massachusetts Buildings Dataset pipeline: 137 training, 4 validation and 10 test images, producing 4,932/144/360 patches. Images were zero-padded from 1500×1500 to 1536×1536 and split into non-overlapping 256×256 patches. All patches were retained in their original source-level splits. RGB inputs use ImageNet normalization; masks are binary.

The comparison reuses the baseline Dataset, preprocessing, metric calculations and diagnostic batch reads. Training starts from the ImageNet encoder and a newly initialized decoder, with the following fixed settings.

Parameter	**Value**
Architecture / encoder	U-Net / ResNet18, ImageNet pretrained
Optimizer / learning rate	Adam / 10^{-3}
Batch size / epochs / seed	8 / 10 / 42
Augmentation / precision	None / full precision
Prediction threshold	0.5
Checkpoint selection	Lowest validation loss for each objective

The main experimental change is the loss function. The test set is used after checkpoint selection, with no threshold or loss-weight tuning.

4. Initial Results

IoU and Dice are computed from global pixel totals over each split, matching the baseline, including background-only and padded pixels. The reference values are the team's saved baseline results; the baseline was not retrained for this milestone.

Split	Metric	BCE baseline	BCE + Dice	Change (pp)
Validation	IoU	0.6338	0.6160	−1.78
Validation	Dice	0.7759	0.7624	−1.35
Test	IoU	0.6898	0.6681	−2.17
Test	Dice	0.8164	0.8010	−1.54

The comparison achieved a test IoU of **0.6681** and a test Dice of **0.8010**. These are lower than the baseline by 2.17 and 1.54 percentage points, respectively. Therefore, adding the chosen Dice term did not improve overlap under the tested configuration.

The best checkpoint was selected at epoch 10. Its validation combined loss was 0.4535 and test combined loss was 0.4446. These values must not be compared numerically with the baseline BCE-only loss because the objectives differ.

5. Implementation Observations

During training, the combined training loss decreased from 0.7458 to 0.5149, and validation IoU increased from 0.5244 to 0.6160. Validation performance fluctuated between epochs; epoch 10 gave both the lowest validation combined loss and the highest validation IoU in this run.

The expected benefit was not observed with equal BCE/Dice coefficients, ten epochs and threshold 0.5. The per-patch soft Dice training term and the global, thresholded evaluation metrics use different aggregation rules, so optimizing the combined objective does not guarantee better reported IoU/Dice. This is a possible explanation, not an experimentally isolated cause.

This is a single run per condition, with only four original validation images. The baseline used a Tesla T4, whereas the comparison used an RTX 5070; hardware and unrecorded differences in the baseline software environment limit strict reproducibility across runs. We do not claim statistical significance or that Dice loss is generally inferior. Threshold sensitivity and detailed failure analysis remain separate team tasks.

6. Reproducibility, Contribution and Repository

The comparison ran on Linux with Python 3.12.3, PyTorch 2.11.0+cu130, torchvision 0.26.0+cu130 and segmentation-models-pytorch 0.5.0, using an NVIDIA GeForce RTX 5070. Training, epoch validation and checkpoint writes took **312.19 seconds** (about 5 minutes 12 seconds). Test evaluation over 360 patches took **1.49 seconds**, including loading, forward passes, loss and metrics; this is not an isolated GPU latency measurement.

The executed notebook is `notebooks/05_comparison_bce_dice.ipynb`; the loss is implemented in `src/losses.py`. Exact setup and run instructions are in `COMPARISON_RUN.md`. Training history, metrics, selected sample identities, environment versions and a run manifest are saved in `results/metrics/`. Selected masks are in `results/predictions/`, and example visualizations are in `figures/comparison_examples.png`. The checkpoint and full validation/test probabilities are retained as separate large artifacts.

Individual contribution: Elena Tuturina implemented and ran the BCE + Dice condition, saved metrics and predictions, documented reproducibility, and prepared the initial comparison and this milestone report.

Git Repository: CV-P22-building-segmentation, experiment/bce-dice-comparison.